

Unit - 6 : Modules

Topic : Modules

- Module Python में एक file होती है जिसमें related functions, classes, variables और code होते हैं।
- Module का use code को organize करने के लिए किया जाता है ताकि उसे reuse कर सकें।
- Python में बहुत से built-in modules available होते हैं और हम अपने own modules भी बना सकते हैं।
- Module use करने के लिए उसे import करना होता है।
- यह code को modular बनाता है, जिससे program को maintain करना आसान हो जाता है।

1. What is a Module ?

- Module एक Python file (.py) होती है जिसमें कुछ useful code लिखा होता है।
- इसमें functions, classes, variables या executable statements हो सकते हैं।
- Module का use code को divide करने और reuse करने के लिए होता है।

Example :

```
# mymodule.py
def greet():
    print("Hello from my module!")
x = 10
```

4. Ways to Import a Module

Method	Syntax	Description
1. Import module	import module	पूरा module import होता है।
2. Import with alias	import module as m	Module को short name देना।
3. Import specific function	from module import function	केवल एक function import करना।
4. Import all	from module import *	Module के सभी members import करना (generally not recommended)।

6. Creating a Module

- हम अपना खुद का module एक .py file बनाकर create कर सकते हैं।
- उस file में functions, classes, variables या कोई भी valid Python code लिख सकते हैं।

Example : (file: calculator.py)

```
def add(a, b):
    return a + b
def subtract(a, b):
    return a - b
pi = 3.14159
```

अब इस module को दूसरी file में import करके use कर सकते हैं।

8. Some Useful Built-in Modules

Module	Use
math	Mathematical functions (sqrt, pi, factorial, etc.)
random	Generate random values
datetime	Work with date and time
os	Operating system related functions
sys	System specific parameters and functions
pathlib	Work with file paths

2. Types of Modules

- Built-in Modules : जो Python के साथ पहले से available होते हैं (जैसे math, random, os, datetime आदि)।
- User-defined Modules : जो हम खुद बनाते हैं (custom modules)।
- Third-party Modules : जो external libraries से install किए जाते हैं (जैसे numpy, pandas आदि)।

3. Importing a Module

- किसी module के functions, classes या variables को use करने के लिए उसे import करना होता है।
- import statement के द्वारा module को program में include किया जाता है।

Example :

```
import math          # पूरा module import करना
import random        # built-in module
import mymodule       # user-defined module
```

5. Using Module Members

- Import करने के बाद हम module के functions, variables और classes को dot (.) operator के साथ access करते हैं।

Example :

```
import math
print(math.sqrt(25))    # 5.0
print(math.pi)         # 3.14159...

import mymodule
mymodule.greet()        # Hello from my module!
print(mymodule.x)       # 10
```

7. Advantages of Modules

- Code reusability बढ़ती है।
- Large programs को छोटे-छोटे modules में divide किया जा सकता है।
- Code को maintain करना आसान हो जाता है।
- Built-in और third-party modules से already available functionality का use कर सकते हैं।
- Collaboration में help मिलती है क्योंकि different developers different modules पर work कर सकते हैं।

Key Points :

- Module एक Python file (.py) होती है।
- Modules code को organize और reuse करने में मदद करते हैं।
- import, from ... import, और as keyword का use किया जाता है।
- Built-in, user-defined और third-party modules होते हैं।
- Module के members को dot (.) operator से access करते हैं।
- Modules program को modular, maintainable और scalable बनाते हैं।

Topic : Standard Modules - sys

- sys Python का एक built-in (standard) module है, जो Python interpreter के साथ interact करने के लिए कई useful functions, variables और objects provide करता है।
- इस module का use system specific parameters और runtime environment की information प्राप्त करने के लिए किया जाता है।
- sys module का use command line arguments, exit करना, path को modify करना, और interpreter से related कई tasks के लिए होता है।

1. Importing sys Module

- sys एक built-in module है, इसलिए इसे अलग से install करने की आवश्यकता नहीं होती।
- इसे use करने के लिए simply import करना होता है।

Example :

```
import sys
print(sys)           # module location दिखाता है
print(sys.__name__)  # output : 'sys'
```

2. Command Line Arguments (sys.argv)

- sys.argv list command line से passed arguments को store करती है।
- पहला element (index 0) हमेशा script का नाम होता है।

Example : (file: demo.py)

```
import sys
print(sys.argv)      # Output (यदि run करें):
                     # ['demo.py', 'arg1', 'arg2']
```

3. Exiting from a Program (sys.exit)

- sys.exit() का use program को terminate करने के लिए किया जाता है।
- इसमें हम exit status भी दे सकते हैं (0 → success, non-zero → error)।

Example :

```
import sys
print("Program is running...")
sys.exit()           # program terminate
print("This will not execute") # यह नहीं चलेगा
```

4. Python Version Information

- sys module से हम current Python version की जानकारी प्राप्त कर सकते हैं।
- यह debugging और compatibility check के लिए useful है।

Example :

```
import sys
print(sys.version)    # full version info
print(sys.version_info) # version tuple
print(sys.platform)   # platform name
```

5. Module Search Path (sys.path)

- sys.path एक list होती है जिसमें उन directories के path होते हैं जहाँ Python modules को search करता है।
- हम इसमें अपने custom module के path को add भी कर सकते हैं।

Example :

```
import sys
print(sys.path)        # search path list
sys.path.append("D:/MyModules") # add new path
```

6. Standard Input, Output, Error

- sys module के द्वारा हम standard input (stdin), output (stdout) और error (stderr) को access कर सकते हैं।
- इनका use advanced I/O handling के लिए किया जाता है।

Example :

```
import sys
print("Output using sys.stdout", file=sys.stdout)
print("Error message", file=sys.stderr)
user_input = sys.stdin.readline() # input पढ़ने के लिए
print("You entered:", user_input)
```

7. Useful Attributes and Functions

Attribute / Function	Description
sys.argv	Command line arguments की list
sys.exit()	Program को terminate करता है
sys.version	Python version की जानकारी
sys.version_info	Version information as tuple
sys.platform	Platform (OS) का नाम
sys.path	Module search path की list
sys.stdin	Standard input stream
sys.stdout	Standard output stream
sys.stderr	Standard error stream
sys.modules	Loaded modules की dictionary

Key Points :

- sys एक built-in (standard) module है।
- इसका use Python interpreter और environment से related information प्राप्त करने के लिए किया जाता है।
- sys.argv command line arguments को store करती है।
- sys.exit() program को terminate करने के लिए होता है।
- sys.path में module search path होते हैं, जिन्हें modify किया जा सकता है।
- यह module debugging, automation और system specific tasks के लिए बहुत useful है।

Topic : Standard Modules - math

- math Python का एक built-in (standard) module है।
- यह module mathematical functions और constants provide करता है।
- math module में trigonometric, logarithmic, power, factorial, rounding आदि कई useful functions होते हैं।
- इसे use करने के लिए पहले import करना होता है। `import math`
- यह mainly numbers (int, float) के साथ work करता है।

Key Points :

- math एक built-in module है।
- Mathematical calculations के लिए use होता है।
- Functions को use करने के लिए import math करते हैं।
- यह केवल numeric data के साथ काम करता है।
- यह module fast और reliable है।

1. Importing math Module

- math module को use करने के लिए import करना आवश्यक होता है।
- इसके बाद हम इसके functions को dot (.) operator द्वारा access करते हैं।

Example :

```
import math
print(math.pi)      # 3.141592653589793
print(math.e)       # 2.718281828459045
```

2. Important Mathematical Constants

Constant	Use	Value
math.pi	Pi (π) का मान	3.141592653589793
math.e	Euler's number	2.718281828459045
math.tau	2 π का मान	6.283185307179586
math.inf	Positive infinity	inf
-math.inf	Negative infinity	-inf
math.nan	Not a Number	nan

3. Commonly Used Functions

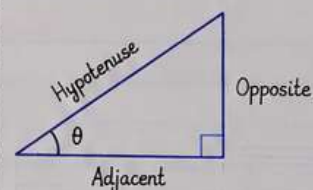
Function	Description	Example	Output
<code>sqrt(x)</code>	Square root	<code>math.sqrt(25)</code>	5.0
<code>pow(x,y)</code>	x raised to y	<code>math.pow(2, 3)</code>	8.0
<code>ceil(x)</code>	Next integer (up)	<code>math.ceil(4.2)</code>	5
<code>floor(x)</code>	Previous integer	<code>math.floor(4.8)</code>	4
<code>factorial(x)</code>	Factorial of x	<code>math.factorial(5)</code>	120
<code>log(x)</code>	Natural logarithm	<code>math.log(10)</code>	2.302...
<code>log10(x)</code>	Base 10 logarithm	<code>math.log10(100)</code>	2.0
<code>sin(x)</code>	Sine (radians)	<code>math.sin(math.pi/2)</code>	1.0
<code>cos(x)</code>	Cosine (radians)	<code>math.cos(0)</code>	1.0
<code>tan(x)</code>	Tangent (radians)	<code>math.tan(math.pi/4)</code>	1.0
<code>radians(x)</code>	Degree to radian	<code>math.radians(180)</code>	3.141...
<code>degrees(x)</code>	Radian to degree	<code>math.degrees(math.pi)</code>	180.0
<code>fabs(x)</code>	Absolute value	<code>math.fabs(-7.5)</code>	7.5
<code>fmod(x,y)</code>	Remainder	<code>math.fmod(7, 3)</code>	1.0

4. Trigonometric Functions (in Radians)

- Trigonometric functions angle को radian में लेते हैं।
- अगर angle degree में हो, तो पहले उसे `radians()` द्वारा convert करते हैं।

Example :

```
import math
angle = 30
rad = math.radians(angle) # degree to radian
print(math.sin(rad))      # 0.5
print(math.cos(rad))      # 0.8660...
print(math.tan(rad))      # 0.5773...
```



$$\sin(\theta) = \frac{\text{Opposite}}{\text{Hypotenuse}}$$

$$\cos(\theta) = \frac{\text{Adjacent}}{\text{Hypotenuse}}$$

$$\tan(\theta) = \frac{\text{Opposite}}{\text{Adjacent}}$$

5. Rounding Functions

- Rounding के लिए math module में `ceil()`, `floor()` और `trunc()` functions available होते हैं।
- ये functions number को integer में convert करने के लिए useful होते हैं।

Function	Description	Example	Output
<code>ceil(x)</code>	Next integer (up)	<code>math.ceil(3.1)</code>	4
<code>floor(x)</code>	Previous integer	<code>math.floor(3.9)</code>	3
<code>trunc(x)</code>	Remove decimal part	<code>math.trunc(3.9)</code>	3

Summary :

- math एक standard module है जो mathematical operations के लिए use होता है।
- import math के बाद इसके functions को dot (.) operator से access करते हैं।
- यह module scientific और general calculations दोनों के लिए बहुत useful है।
- सही function का चयन करके हम अलग-अलग mathematical problems को आसानी से solve कर सकते हैं।

6. Other Useful Functions

- math module में और भी कई useful functions होते हैं।
- ये functions different mathematical operations के लिए इस्तेमाल किए जाते हैं।

Function	Description	Example	Output
<code>isfinite(x)</code>	Check finite number	<code>math.isfinite(10)</code>	True
<code>isinf(x)</code>	Check infinity	<code>math.isinf(math.inf)</code>	True
<code>isnan(x)</code>	Check NaN	<code>math.isnan(math.nan)</code>	True
<code>copysign(x,y)</code>	Sign copy	<code>math.copysign(3, -1)</code>	-3.0
<code>hypot(x,y)</code>	$\sqrt{x^2 + y^2}$	<code>math.hypot(3, 4)</code>	5.0
<code>gcd(x,y)</code>	Greatest common divisor	<code>math.gcd(16, 24)</code>	8
<code>lcm(x,y)</code>	Least common multiple	<code>math.lcm(4, 6)</code>	12

Topic : Standard Modules - time

- time Python का एक built-in (standard) module है जो time से related functions provide करता है।
- इसमें module का use current time प्राप्त करने, time को format करने, program को delay देने, performance measure करने और time conversion आदि के लिए किया जाता है।
- यह module system clock के साथ interact करता है और time से related कई useful functions देता है।
- time module में time data generally seconds (epoch) के रूप में represent होता है।

1. Importing time Module

- time module already Python में available होता है, इसलिए हमें इसे install करने की आवश्यकता नहीं होती है।
- इसे use करने के लिए simply import करते हैं।

Example :

```
import time
print("Time module imported successfully!")
```

2. Current Time Functions

- current time प्राप्त करने के लिए time() और ctime() functions का use किया जाता है।
- time() वर्तमान time को seconds (epoch) में return करता है।
- ctime() उस time को readable format में convert करता है।

Example :

```
import time
t = time.time()
print(t) # 1738664920.123 (epoch time)
print(time.ctime(t)) # Tue Feb 4 21:48:40 2025
```

3. Working with local time

- localtime() function epoch time को local time structure (struct_time) में convert करता है।
- यह year, month, day, hour, minute, second आदि की information देता है।

Example :

```
import time
t = time.time()
local = time.localtime(t)
print(local) # time.struct_time(tm_year=2025, ...)
print(local.tm_year) # 2025
print(local.tm_mon) # 2
print(local.tm_mday) # 4
```

struct_time (local time structure)

tm_year	Year (जैसे 2025)
tm_mon	Month (1 - 12)
tm_mday	Day (1 - 31)
tm_hour	Hour (0 - 23)
tm_min	Minute (0 - 59)
tm_sec	Second (0 - 59)
tm_wday	Weekday (0 = Monday)
tm_yday	Day of year (1 - 366)
tm_isdst	Daylight saving flag

4. Formatting Time

- strftime() function से हम time को अपनी आवश्यकता के अनुसार format में convert कर सकते हैं।
- इसके लिए format codes का use किया जाता है।

Example :

```
import time
now = time.localtime()
print(time.strftime("%Y-%m-%d", now)) # 2025-02-04
print(time.strftime("%d/%m/%Y %H:%M:%S", now))
# 04/02/2025 21:48:40
```

Common Format Codes

Code	Meaning
%Y	4 digit year (2025)
%m	Month (01 - 12)
%d	Day (01 - 31)
%H	Hour (00 - 23)
%M	Minute (00 - 59)
%S	Second (00 - 59)
%A	Weekday name (Tuesday)
%B	Month name (February)

5. Pausing Execution (Sleep)

- sleep() function program को कुछ समय के लिए pause करता है।
- इसमें time seconds में दिया जाता है।

Example :

```
import time
print("Start")
time.sleep(2) # 2 seconds pause
print("End (after 2 seconds)")
```

6. Measuring Execution Time

- किसी code या program के execution time को measure करने के लिए time() function का use किया जाता है।

Example :

```
import time
start = time.time()
for i in range(1000000):
    pass
end = time.time()
print("Execution time:", end - start, "seconds")
```

7. Other Useful Functions

Function	Description
time.time()	Current time in seconds (epoch)
time.ctime()	Convert time to readable string
time.localtime()	Convert to local time structure
time.strftime()	Format time as string
time.sleep()	Pause execution for given seconds
time.gmtime()	Convert to UTC time structure
time.perf_counter()	High resolution timer (for performance measurement)

Key Points :

- time module एक built-in module है।
- time() current time seconds में देता है।
- ctime() readable format में time देता है।
- strftime() से custom format बनाया जा सकता है।
- sleep() program को delay करने के लिए use होता है।
- time module performance measurement के लिए भी useful है।
- यह module कई real-life applications (logging, scheduling, etc.) में use किया जाता है।

Topic : The dir() Function

- `dir()` Python का एक built-in function है, जो किसी object के सभी available attributes (जैसे variables, functions, methods, classes आदि) की list return करता है।
- यह function introspection के लिए उपयोगी होता है, क्योंकि इसके द्वारा हम किसी module, class, object या current namespace में मौजूद names को देख सकते हैं।
- `dir()` हमेशा sorted (alphabetical order) में एक list return करता है।

1. Syntax

```
dir([object])
```

- `object (optional)` : वो object जिसका attributes देखना है।
- अगर object नहीं दिया जाता, तो `dir()` current local scope के names की list return करता है।

2. Return Value

- `dir()` हमेशा एक list return करता है जिसमें strings के रूप में सभी valid names होते हैं।
- इन names में variables, functions, classes, methods और other attributes शामिल होते हैं।

3. Common Use Cases

- किसी module में उपलब्ध functions और variables देखने के लिए।
- किसी object के methods और attributes जानने के लिए।
- Exploring Python library/module के contents के लिए।
- Interactive Python shell में debugging और learning के लिए।

4. Examples

a) Current Local Scope

```
>>> a = 10
>>> b = 'hello'
>>> dir()
```

Output (partial) :

```
['__annotations__',
'__builtins__',
'__doc__',
'a', 'b', ...]
```

Note : `dir()` बिना argument के current scope के सभी names दिखाता है।

b) On a Module

```
>>> import math
>>> dir(math)
```

Output (partial) :

```
['__doc__', '__file__',
'acos', 'asin', 'atan',
'ceil', 'cos', 'sin', ...]
```

Note : इससे `math` module के सभी functions और attributes की list प्राप्त होती है।

c) On a Built-in Type

```
>>> dir(list)
```

Output (partial) :

```
['__add__', '__class__',
'__contains__', 'append',
'clear', 'copy', 'count', ...]
```

Note : `dir(list)` `list` class के सभी methods दिखाता है।

d) On a User-defined Object

```
class Student:
    def __init__(self, name, age):
        self.name = name
        self.age = age
    def show(self):
        print(self.name, self.age)
s = Student("Rahul", 20)
print(dir(s))
```

Output (partial) :

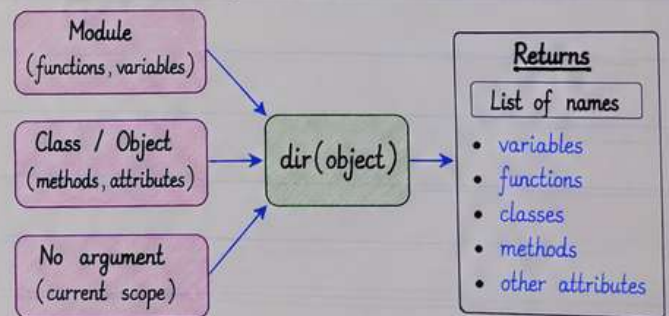
```
['__class__', '__delattr__', '__dict__', '__dir__',
'__doc__', '__eq__', 'age', 'name', 'show', ...]
```

Note : इसमें object के attributes (name, age) और methods (show) भी दिखाई दे रहे हैं।

5. Important Points

- `dir()` केवल valid attributes की list दिखाता है, उनके values नहीं।
- यह list हमेशा alphabetical order में होती है।
- `dir()` अलग-अलग प्रकार के objects (module, class, instance, function आदि) पर काम करता है।
- यह private और special methods (जैसे `__init__`, `__str__`, `__dict__` आदि) भी दिखाता है।
- यह debugging, exploration और learning के लिए बहुत useful है।

6. Visualization



7. Example Output in Python Shell

```
>>> import time
>>> dir(time)
['CLOCK_BOOTTIME', 'CLOCK_MONOTONIC', 'ctime',
'daylight', 'gmtime', 'localtime', 'mktime', 'sleep',
'strftime', 'time', 'timezone', 'tzname', ...]
```

Key Takeaways :

- `dir()` Python का built-in function है।
- यह किसी object के सभी valid attributes की list return करता है।
- यह values नहीं, केवल names दिखाता है।
- यह introspection, debugging और exploration के लिए बहुत useful है।